QB: Chapter 3.1 — Graphs & Binary Trees

Unit 3 | 4 × 2-mark | 5 × 5-mark | 4 × 10-mark | **Total: 13 questions**
Lectures: L30–L35 | BT Levels: BT1, BT2, BT3, BT5 | COs: CO2, CO3, CO4, CO5

Section A: 2-Mark Questions

Q1. [BT1 | CO2] Define a graph G = (V, E). Distinguish between a *directed graph (digraph)* and an *undirected graph* with respect to edge representation and degree definition.

Q2. [BT1 | CO2] Define a *binary tree*. State the maximum number of nodes at level k (root at level 0). For a perfect binary tree of height h, state the total number of nodes.

Q3. [BT2 | CO2] Explain the difference between *adjacency matrix* and *adjacency list* representations of a graph with V vertices and E edges. Compare space complexity and the time to check if edge (u, v) exists under each representation.

Q4. [BT2 | CO3] Describe the BFS and DFS graph traversal strategies. What data structure does each require? State one problem that BFS is best suited for and one that DFS is best suited for.

Section B: 5-Mark Questions

Q5. [BT3 | CO2] Consider graph G with vertices {A, B, C, D, E} and undirected edges {(A,B), (A,C), (B,D), (C,D), (D,E)}.

(a) Construct the adjacency matrix for G. (b) Construct the adjacency list for G. (c) Compare the space used by each representation for this specific graph and state when each is preferred.

Q6. [BT5 | CO2, CO3] Perform BFS on graph G from Q5 starting from vertex A. Use a queue.

(a) Show the queue state and visited set at the start of each iteration. (b) List the BFS traversal order. (c) State the BFS tree edges (edges used to first discover each vertex). Explain why BFS finds the shortest path (in terms of number of edges) in an unweighted graph.

Q7. [BT5 | CO2, CO3, CO5] Perform DFS on graph G from Q5 starting from vertex A. Use an explicit stack.

(a) Show the stack state and visited set at each step. (b) List the DFS traversal order. (c) Compare the DFS and BFS traversal orders for this graph. Explain which algorithm finds the shortest unweighted path between A and E, and why DFS cannot guarantee this.

Q8. [BT3 | CO2] Consider a binary tree with root 10, where 10's left child is 5 (with children 3 and 7) and 10's right child is 15 (with children 12 and 20).

(a) List the nodes in inorder, preorder, and postorder traversal sequences. (b) Give the level-order (BFS) traversal sequence. (c) Identify all leaves, the height of the tree, and the depth of node 7.

Q9. [BT5 | CO2, CO3] Implement iterative *inorder traversal* of a binary tree using an explicit stack. Use the tree from Q8.

(a) Trace the algorithm step by step. After each operation (push, pop, visit, move right), show the stack contents and the result list built so far. (b) Compare the iterative approach with recursive inorder traversal. Identify the implicit data structure the recursive approach uses. Analyse the space complexity of each approach for a balanced tree of n nodes.

Section C: 10-Mark Questions

Q10. [BT5 | CO2, CO3] *(Experiments E7 & E12 — Number of Islands, LeetCode #200)*

The "Number of Islands" problem counts connected regions of '1's in a binary 2D grid, where '1' represents land and '0' represents water.

(a) Model the grid as a graph where each land cell is a vertex and adjacent land cells share an edge. Design both a BFS-based and a DFS-based algorithm to count distinct islands. Write complete Python implementations for each. Explain the role of the "visited" marking.

(b) Trace the DFS algorithm on the following grid (row-major, 0-indexed). Show the order of cell visits and indicate at which point each island is counted:

```
Grid:
[ [1, 1, 0, 0, 0],
  [1, 1, 0, 0, 0],
  [0, 0, 1, 0, 0],
  [0, 0, 0, 1, 1] ]
```

(c) Analyse time complexity (O(m $\times$ n)) and space complexity of both BFS and DFS solutions. For an extremely large grid (e.g., $10^6 \times 10^6$), discuss which traversal strategy is preferable with respect to maximum stack/queue depth and practical memory constraints.

Q11. [BT5 | CO3, CO4] *(Experiment E8 — Network Delay Time, LeetCode #743)*

In this problem, a signal is sent from a source node k through a directed, weighted network of n nodes. Find the time for all nodes to receive the signal.

(a) Explain why this is equivalent to the single-source shortest path (SSSP) problem. Implement Dijkstra's algorithm using a min-priority queue (min-heap). Write the complete Python solution including graph construction from the edge list.

(b) Trace Dijkstra's algorithm on the network: n = 4, edges = [(2,1,1), (2,3,1), (3,4,1)], source = 2 (1-indexed nodes). Show the priority queue contents and the distance array `dist[]` after each extraction step. State the final answer.

(c) Derive the time complexity of Dijkstra's algorithm with a binary min-heap as O((V + E) log V). Explain with a concrete counterexample why Dijkstra's algorithm gives incorrect results when negative-weight edges are present, and identify which algorithm should be used instead.

Q12. [BT5 | CO2, CO3] *(Experiment E10 — Course Schedule, LeetCode #207)*

The Course Schedule problem determines whether it is possible to complete all courses given a list of prerequisite pairs — equivalently, whether the directed prerequisite graph contains a cycle.

(a) Explain how cycle detection in a directed graph solves this problem. Implement DFS-based cycle detection using three-colour marking: white (unvisited), grey (in current DFS path), black (fully processed). Write the complete Python function `can_finish(numCourses, prerequisites)`.

(b) Trace the algorithm on prerequisites = [[1, 0], [0, 1]] (course 0 requires course 1, and course 1 requires course 0). Identify the exact step at which the grey → grey back-edge is detected.

(c) Implement *Kahn's algorithm* (BFS-based topological sort with in-degree counting) as an alternative cycle detection method. Compare the DFS and Kahn's approaches on: time complexity, space complexity, and whether they also produce a valid course order when no cycle exists.

Q13. [BT5 | CO2, CO3] *(Experiment E11 — Maximum Depth of Binary Tree, LeetCode #104)*

The Maximum Depth problem asks for the number of nodes along the longest root-to-leaf path in a binary tree.

(a) Implement both a recursive DFS solution and an iterative BFS (level-order) solution. For the DFS solution, clearly state the base case, the recursive case, and how the maximum is computed. For the BFS solution, explain how to track the number of levels.

(b) Trace both solutions on the tree: root = 3, root.left = 9, root.right = 20, 20.left = 15, 20.right = 7. For DFS, show the call stack at each recursive call. For BFS, show the queue state at the start of each level.

(c) Derive the time complexity O(n) and space complexity for each approach. For a perfectly balanced tree of height h, compare maximum call stack depth (recursive) versus maximum queue size (iterative). Identify which has better space characteristics for a *degenerate* (skewed) tree of n nodes and explain why.